

FreeRTOS Scheduler Project

Generated by Doxygen 1.17.0

1 Project Documentation

1.1 Implementation Summary

Based on the project requirements, the **Periodic Task Layer (PTL)** has been successfully designed and integrated on top of FreeRTOS. Below is a summary of the architectural and functional components implemented:

- **Kernel Patching & Portability:** Developed a thin, minimally intrusive abstraction layer that interfaces with the FreeRTOS kernel hooks without altering core scheduler preemption or Round-Robin semantics.
- **Synchronized Lifecycle Management:** Implemented an initialization pipeline via `Init()` and a unified `Start()` entrypoint, enforcing the system requirement that all registered periodic tasks synchronize their phase and start simultaneously at a shared baseline time t_0 .
- **Automated Task Wrapper Loop:** Created a specialized runtime wrapper that abstractly encapsulates task execution. The programmer only implements the functional body of the task loop, while the PTL automatically manages the execution cadence and system-level boundaries.
- **Real-Time Constraint Monitors:**
 - Integrated automated execution profiling within a job completion hook (`JobComplete()`) to benchmark Finish Time (F_k) against the relative Deadline ($F_k > R_k + D$), triggering `DEADLINE_MISS` logging.
 - Embedded release-time monitors evaluated at every new instance (R_{k+1}) to detect active overflows where the preceding job failed to yield, triggering a `PERIOD_OVERRUN`.
- **Dynamic Overrun Mitigation Policies:** Implemented and integrated three deterministic recovery strategies configurable globally or per-task:
 - **SKIP:** Suppresses the release of job $k + 1$, granting the late job k the required headroom to safely complete execution.
 - **KILL:** Forcefully terminates/suspends the violating job k immediately upon overrun detection to instantly instantiate the fresh job $k + 1$.
 - **CATCH_UP:** Commits a parallel instance release for job $k + 1$, maintaining strict nominal timing cadence while flagging the tardy job as a failure.
- **Centralized Configuration Interface:** Deployed a structured `SchedulerConfig` object capable of statically or dynamically registering global metadata (max tasks, tracing status, default fallback policies) alongside detailed per-task configurations (name, priority, stack size, period T , and explicit/implicit deadlines D).

2 Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

Macros (Trace Hooks)

??

3 Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

SubFrame_t	One slot inside a major frame; reserved for future sub-frame metadata	??
TestEvent_t	Per-test observation record	??
TimelineConfig_t	Global scheduler state, passed to vConfigureScheduler() once at boot	??
TimelineTaskConfig_t	Per-task configuration supplied by the application in main.c	??

4 File Index

4.1 File List

Here is a list of all files with brief descriptions:

FreeRTOSConfig.h	Custom configurations and modifications for FreeRTOS	??
main.c	Main application entry point for the time-triggered scheduler tests	??
scheduler.c	Implementation of the time-triggered scheduler extension for FreeRTOS	??
scheduler.h	Time-triggered scheduler extension for FreeRTOS	??
tasks.c		??
test_scheduler.c	Automated regression test suite for the timeline-based scheduler	??
test_scheduler.h	Public API for the timeline-scheduler regression test suite	??
uart.c		??

5 Topic Documentation

5.1 Macros (Trace Hooks)

Macros to monitor the scheduler's context switch.

Macros

- `#define traceTASK\_SWITCHED\_IN\(\)`
Hook triggered at the exact moment a task is loaded onto the CPU.
- `#define traceTASK\_SWITCHED\_OUT\(\)`
Hook triggered at the exact moment a task is removed from the CPU.

Functions

- void `UART_printf_ISR` (const char *s,...)
UART print routine safe for ISR calls.

5.1.1 Detailed Description

Macros to monitor the scheduler's context switch.

•

`UART_printf_ISR` is safe with respect to ISRs (no critical sections, no FreeRTOS API calls). Both macros are activated within the context-switch path, which is effectively an ISR context, so only ISR-safe calls are allowed here.

5.1.2 Macro Definition Documentation

`traceTASK_SWITCHED_IN`

```
#define traceTASK_SWITCHED_IN()
```

Value:

```
do {
    UART_printf_ISR( "[ %d ms ] %s start\n",
                    ( int ) xTickCount,
                    pxCurrentTCB->pcTaskName );
} while( 0 )
```

```
\\
\\
\\
```

Hook triggered at the exact moment a task is loaded onto the CPU.

`traceTASK_SWITCHED_OUT`

```
#define traceTASK_SWITCHED_OUT()
```

Value:

```
do {
    UART_printf_ISR( "[ %d ms ] %s end\n",
                    ( int ) xTickCount,
                    pxCurrentTCB->pcTaskName );
} while( 0 )
```

```
\\
\\
\\
```

Hook triggered at the exact moment a task is removed from the CPU.

5.1.3 Function Documentation

UART_printf_ISR()

```
void UART_printf_ISR (
    const char * s,
    ...) [extern]
```

UART print routine safe for ISR calls.

Parameters

<code>s</code>	Format string for the output.
----------------	-------------------------------

6 Data Structure Documentation

6.1 SubFrame_t Struct Reference

One slot inside a major frame; reserved for future sub-frame metadata.

```
#include <scheduler.h>
```

Data Fields

- `uint32_t id`

6.1.1 Detailed Description

One slot inside a major frame; reserved for future sub-frame metadata.

6.1.2 Field Documentation

id

```
uint32_t SubFrame_t::id
```

The unique identifier for the sub-frame.

The documentation for this struct was generated from the following file:

- [scheduler.h](#)

6.2 TestEvent_t Struct Reference

Per-test observation record.

```
#include <test_scheduler.h>
```

Data Fields

- [EventType_t](#) eType
- char [pcTaskName](#) [configMAX_TASK_NAME_LEN]
- [TickType_t](#) xTick
- [uint32_t](#) ulFrameTick

6.2.1 Detailed Description

Per-test observation record.

The test-probe callbacks fill one of these for every task event that occurs during a test run. The verifier function for each test walks the log and checks invariants.

6.2.2 Field Documentation

eType

```
EventType_t TestEvent_t::eType
```

The recorded event type.

pcTaskName

```
char TestEvent_t::pcTaskName[configMAX_TASK_NAME_LEN]
```

The name of the task associated with the event.

ulFrameTick

```
uint32_t TestEvent_t::ulFrameTick
```

The tick offset inside the current major frame.

xTick

```
TickType_t TestEvent_t::xTick
```

The absolute xTaskGetTickCount() at the moment.

The documentation for this struct was generated from the following file:

- [test_scheduler.h](#)

6.3 TimelineConfig_t Struct Reference

Global scheduler state, passed to [vConfigureScheduler\(\)](#) once at boot.

```
#include <scheduler.h>
```

Data Fields

- [TimelineTaskConfig_t](#) * tasks
- uint32_t tasks_num
- uint32_t num_sub_frames
- uint32_t major_frame_period
- uint32_t sub_frame_period
- uint32_t current_tick
- uint32_t current_sub_frame

6.3.1 Detailed Description

Global scheduler state, passed to [vConfigureScheduler\(\)](#) once at boot.

6.3.2 Field Documentation

current_sub_frame

```
uint32_t TimelineConfig_t::current_sub_frame
```

Current sub-frame index (0 ... num_sub_frames - 1).

current_tick

```
uint32_t TimelineConfig_t::current_tick
```

Current tick offset (0 ... major_frame_period - 1).

major_frame_period

```
uint32_t TimelineConfig_t::major_frame_period
```

Must equal MAJOR_FRAME_DURATION.

num_sub_frames

```
uint32_t TimelineConfig_t::num_sub_frames
```

Derived from MAJOR_FRAME / SUB_FRAME.

sub_frame_period

```
uint32_t TimelineConfig_t::sub_frame_period
```

Must equal SUB_FRAME_DURATION.

tasks

```
TimelineTaskConfig_t* TimelineConfig_t::tasks
```

Pointer to the task-config array.

tasks_num

```
uint32_t TimelineConfig_t::tasks_num
```

Number of entries in that array.

The documentation for this struct was generated from the following file:

- [scheduler.h](#)

6.4 TimelineTaskConfig_t Struct Reference

Per-task configuration supplied by the application in [main.c](#).

```
#include <scheduler.h>
```

Data Fields

- const char * [task_name](#)
- TaskFunction_t [function](#)
- TaskType_t type
- uint32_t [ulStart_time](#)
- uint32_t [ulEnd_time](#)
- uint32_t [ulSubframe_id](#)
- TaskHandle_t [xHandle](#)
- uint8_t [is_completed](#)

6.4.1 Detailed Description

Per-task configuration supplied by the application in [main.c](#).

The scheduler reads this table; the application must not modify it after [vConfigureScheduler\(\)](#) has been called.

6.4.2 Field Documentation

function

`TaskFunction_t TimelineTaskConfig_t::function`

The task entry point.

is_completed

`uint8_t TimelineTaskConfig_t::is_completed`

pdTRUE once the task has finished or been killed.

task_name

`const char* TimelineTaskConfig_t::task_name`

Human-readable name (configMAX_TASK_NAME_LEN).

type

`TaskType_t TimelineTaskConfig_t::type`

Task priority classification (HARD_RT or SOFT_RT).

ulEnd_time

`uint32_t TimelineTaskConfig_t::ulEnd_time`

Deadline tick offset; HRT is killed if still running at this point.

ulStart_time

`uint32_t TimelineTaskConfig_t::ulStart_time`

Tick offset inside the major frame to activate.

ulSubframe_id

`uint32_t TimelineTaskConfig_t::ulSubframe_id`

Sub-frame this task belongs to.

xHandle

TaskHandle_t TimelineTaskConfig_t::xHandle

FreeRTOS task handle, set by xTaskCreate().

The documentation for this struct was generated from the following file:

- [scheduler.h](#)

7 File Documentation

7.1 FreeRTOSConfig.h File Reference

Custom configurations and modifications for FreeRTOS.

Macros

- #define [configUSE_TRACE_FACILITY](#) 1
Enables FreeRTOS tracing facility.
- #define [configUSE_TICK_HOOK](#) 1
Enables the system tick hook function.
- #define [configUSE_TIME_SLICING](#) 0
Enables or disables time-slicing.
- #define [configKERNEL_INTERRUPT_PRIORITY](#) (255)
Minimum interrupt priority (kernel interrupts).
- #define [configASSERT](#)(x)
Assertion macro to validate parameters and internal states.
- #define [configMAX_SYSCALL_INTERRUPT_PRIORITY](#) (4)
Maximum interrupt priority from which FreeRTOS APIs can be called (safe API).
- #define [configUSE_PORT_OPTIMISED_TASK_SELECTION](#) 0
Enables optimized architectural methods for task selection.
- #define [configENABLE_BACKWARD_COMPATIBILITY](#) 0
Enables backward compatibility with data types of older versions.
- #define [traceTASK_SWITCHED_IN](#)()
Hook triggered at the exact moment a task is loaded onto the CPU.
- #define [traceTASK_SWITCHED_OUT](#)()
Hook triggered at the exact moment a task is removed from the CPU.

Software Timer Configurations

-
- #define [configUSE_TIMERS](#) 1
Enables the use of software timers.
- #define [configTIMER_TASK_PRIORITY](#) (configMAX_PRIORITIES - 1)
Defines the priority of the timer daemon task.
- #define [configTIMER_QUEUE_LENGTH](#) 10
Defines the length of the timer command queue.

Testing and Debugging

-
- #define `configRUN_ADDITIONAL_TESTS` 1
Enables the execution of additional tests.
- #define `configSTREAM_BUFFER_TRIGGER_LEVEL_TEST_MARGIN` 4
Defines the margin for the stream buffer trigger level in tests.

Application Task Priorities

-
- #define `intqHIGHER_PRIORITY` (`configMAX_PRIORITIES` - 5)
- #define `bktPRIMARY_PRIORITY` (`configMAX_PRIORITIES` - 3)
- #define `bktSECONDARY_PRIORITY` (`configMAX_PRIORITIES` - 4)

Functions

- void `UART_printf_ISR` (const char *s,...)
UART print routine safe for ISR calls.

7.1.1 Detailed Description

Custom configurations and modifications for FreeRTOS.

7.1.2 Macro Definition Documentation

bktPRIMARY_PRIORITY

```
#define bktPRIMARY_PRIORITY ( configMAX_PRIORITIES - 3 )
```

bktSECONDARY_PRIORITY

```
#define bktSECONDARY_PRIORITY ( configMAX_PRIORITIES - 4 )
```

configASSERT

```
#define configASSERT(  
    x)
```

Value:

```
if( ( x ) == 0 ) while( 1 )
```

Assertion macro to validate parameters and internal states.

Parameters

<i>x</i>	Condition to verify. If it is 0 (false), it blocks execution in an infinite loop.
----------	---

configENABLE_BACKWARD_COMPATIBILITY

```
#define configENABLE_BACKWARD_COMPATIBILITY 0
```

Enables backward compatibility with data types of older versions.

configKERNEL_INTERRUPT_PRIORITY

```
#define configKERNEL_INTERRUPT_PRIORITY ( 255 )
```

Minimum interrupt priority (kernel interrupts).

Note

QEMU does not model priority bits completely, so all eight are used.

configMAX_SYSCALL_INTERRUPT_PRIORITY

```
#define configMAX_SYSCALL_INTERRUPT_PRIORITY ( 4 )
```

Maximum interrupt priority from which FreeRTOS APIs can be called (safe API).

Warning

configMAX_SYSCALL_INTERRUPT_PRIORITY must not be zero.

configRUN_ADDITIONAL_TESTS

```
#define configRUN_ADDITIONAL_TESTS 1
```

Enables the execution of additional tests.

configSTREAM_BUFFER_TRIGGER_LEVEL_TEST_MARGIN

```
#define configSTREAM_BUFFER_TRIGGER_LEVEL_TEST_MARGIN 4
```

Defines the margin for the stream buffer trigger level in tests.

configTIMER_QUEUE_LENGTH

```
#define configTIMER_QUEUE_LENGTH 10
```

Defines the length of the timer command queue.

configTIMER_TASK_PRIORITY

```
#define configTIMER_TASK_PRIORITY ( configMAX_PRIORITIES - 1 )
```

Defines the priority of the timer daemon task.

configUSE_PORT_OPTIMISED_TASK_SELECTION

```
#define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
```

Enables optimized architectural methods for task selection.

Note

Set to 0 because QEMU does not model the CLZ instruction.

configUSE_TICK_HOOK

```
#define configUSE_TICK_HOOK 1
```

Enables the system tick hook function.

•

FIX: must be 1 for [vApplicationTickHook\(\)](#) to be called. The original file set it to 0 with a comment saying that the scheduler would use it – which is contradictory. Our scheduler logic is located in the [tasks.c](#) patch (`xTaskIncrementTick`), so the hook itself is a no-op, but setting it to 1 avoids linking issues if it is ever needed in the future.

configUSE_TIME_SLICING

```
#define configUSE_TIME_SLICING 0
```

Enables or disables time-slicing.

•

Disables time-slicing: SRT tasks run only when no HRT task is ready, and the Idle task runs only when no other task is ready.

configUSE_TIMERS

```
#define configUSE_TIMERS 1
```

Enables the use of software timers.

configUSE_TRACE_FACILITY

```
#define configUSE_TRACE_FACILITY 1
```

Enables FreeRTOS tracing facility.

•

FIX: configUSE_TRACE_FACILITY was defined twice (first 0, then 1 via #undef + redefinition at the bottom of the file). Consolidated here as a single definition set to 1, which is required for the traceTASK_SWITCHED_IN / traceTASK_SWITCHED_OUT macros defined below.

intqHIGHER_PRIORITY

```
#define intqHIGHER_PRIORITY ( configMAX_PRIORITIES - 5 )
```

7.2 FreeRTOSConfig.h

[Go to the documentation of this file.](#)

```
00001
00005
00014 #define configUSE_TRACE_FACILITY 1
00015
00025 #define configUSE_TICK_HOOK 1
00026
00031 #define configUSE_TIMERS 1
00033 #define configTIMER_TASK_PRIORITY ( configMAX_PRIORITIES - 1 )
00035 #define configTIMER_QUEUE_LENGTH 10
00037
00044 #define configUSE_TIME_SLICING 0
00045
00046 /* This demo makes use of one or more sample statistics formatting functions. */
00047
00053 #define configKERNEL_INTERRUPT_PRIORITY ( 255 )
00054
00055 #ifndef __IASMARM__
00061 #define configASSERT( x ) if( ( x ) == 0 ) while( 1 )
00062 #endif
00063
00069 #define configMAX_SYSCALL_INTERRUPT_PRIORITY ( 4 )
00070
00076 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
00077
00082 #define configRUN_ADDITIONAL_TESTS 1
00084 #define configSTREAM_BUFFER_TRIGGER_LEVEL_TEST_MARGIN 4
00086
00090 #define intqHIGHER_PRIORITY ( configMAX_PRIORITIES - 5 )
00091 #define bktPRIMARY_PRIORITY ( configMAX_PRIORITIES - 3 )
00092 #define bktSECONDARY_PRIORITY ( configMAX_PRIORITIES - 4 )
00094
00099 #define configENABLE_BACKWARD_COMPATIBILITY 0
00100
00109
00114 extern void UART_printf_ISR( const char *s, ... );
00115
00120 #define traceTASK_SWITCHED_IN()
00121 do {
00122     UART_printf_ISR( "[ %d ms ] %s start\n",
```

```
\
\
\
```

```

00123             ( int ) xTickCount,                \
00124             pxCurrentTCB->pcTaskName );        \
00125     } while( 0 )
00126
00131 #define traceTASK_SWITCHED_OUT()              \
00132     do {                                       \
00133         UART_printf_ISR( "[ %d ms ] %s end\n", \
00134             ( int ) xTickCount,               \
00135             pxCurrentTCB->pcTaskName );        \
00136     } while( 0 )
00137 /* End of TraceMacros group */
00139
00140 #endif /* FREERTOS_CONFIG_H */

```

7.3 main.c File Reference

Main application entry point for the time-triggered scheduler tests.

```

#include "FreeRTOS.h"
#include "task.h"
#include "uart.h"
#include "test_scheduler.h"

```

Functions

- `int main (void)`
Main application entry point.

7.3.1 Detailed Description

Main application entry point for the time-triggered scheduler tests.

7.3.2 Function Documentation

main()

```

int main (
    void )

```

Main application entry point.

Initializes the hardware (UART), sets up the supervisor task that sequences the scheduler tests, and starts the FreeRTOS scheduler.

Returns

`int` Returns 0. The program should never reach this point as the scheduler takes over execution.

7.4 scheduler.c File Reference

Implementation of the time-triggered scheduler extension for FreeRTOS.

```

#include "scheduler.h"
#include "uart.h"

```

Functions

- void `vConfigureScheduler` (`TimelineConfig_t *cfg`)
Configures and initializes the time-triggered scheduler.
- void `vApplicationTickHook` (void)
Application Tick Hook for the time-triggered scheduler.

Variables

- `TimelineConfig_t * pxTimelineState` = NULL
Global pointer to the timeline state.
- `TaskHandle_t xFrameManagerHandle` = NULL
Handle of the Frame Manager task.

7.4.1 Detailed Description

Implementation of the time-triggered scheduler extension for FreeRTOS.

FreeRTOS V202212.00 Copyright (C) 2020 Amazon.com, Inc. or its affiliates. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

<https://www.FreeRTOS.org> <https://github.com/FreeRTOS>

7.4.2 Function Documentation

`vApplicationTickHook()`

```
void vApplicationTickHook (
    void )
```

Application Tick Hook for the time-triggered scheduler.

Called from `xTaskIncrementTick()` on every tick when `configUSE_TICK_HOOK == 1`.

Note

The primary scheduling logic (wake/kill HRT, wake Frame Manager) lives in the `tasks.c` patch inside `xTaskIncrementTick()` so that it has direct access to kernel internals (`xSwitchRequired`, `list ops`). This hook is intentionally empty; it exists only to satisfy the linker requirement when `configUSE_TICK_HOOK == 1`.

vConfigureScheduler()

```
void vConfigureScheduler (
    TimelineConfig_t * cfg)
```

Configures and initializes the time-triggered scheduler.

Initializes the time-triggered scheduler.

This function validates the configuration, cleans up any previous test states, spawns the Frame Manager task, and creates all application tasks (initially suspended).

Parameters

<i>cfg</i>	Pointer to the timeline configuration structure.
------------	--

Configures and initializes the time-triggered scheduler.

Call once from [main\(\)](#), before [vTaskStartScheduler\(\)](#). Parses *cfg*, creates all FreeRTOS tasks (suspended), and spawns the Frame Manager task that resets the timeline at every major-frame boundary.

Parameters

<i>cfg</i>	Pointer to the global timeline configuration structure.
------------	---

7.4.3 Variable Documentation**pxTimelineState**

```
TimelineConfig_t* pxTimelineState = NULL
```

Global pointer to the timeline state.

Global state for the timeline scheduler configuration.

Set once by [vConfigureScheduler\(\)](#) and then read (never written) by the tick-hook logic inside [tasks.c](#). Declared extern there so the linker connects both translation units.

Global pointer to the timeline state.

xFrameManagerHandle

```
TaskHandle_t xFrameManagerHandle = NULL
```

Handle of the Frame Manager task.

Handle for the Frame Manager Task.

The tick hook resumes it at every major-frame boundary (`now == 0`) so it can reset HRT tasks. Also declared extern in [tasks.c](#).

Handle of the Frame Manager task.

7.5 scheduler.h File Reference

Time-triggered scheduler extension for FreeRTOS.

```
#include "FreeRTOS.h"
#include "task.h"
```

Data Structures

- struct [SubFrame_t](#)
One slot inside a major frame; reserved for future sub-frame metadata.
- struct [TimelineTaskConfig_t](#)
Per-task configuration supplied by the application in [main.c](#).
- struct [TimelineConfig_t](#)
Global scheduler state, passed to [vConfigureScheduler\(\)](#) once at boot.

Macros

Timing Constants

Adjust to match your target CPU / tick rate.

- #define [MAJOR_FRAME_DURATION](#) ((TickType_t) 100)
- #define [SUB_FRAME_DURATION](#) ((TickType_t) 10)
- #define [NUM_SUB_FRAMES](#) ([MAJOR_FRAME_DURATION](#) / [SUB_FRAME_DURATION](#))
- #define [MAX_SRT_PER_SUBFRAME](#) (2)

Enumerations

- enum [TaskType_t](#) { [HARD_RT](#) , [SOFT_RT](#) }
Task classification for the time-triggered scheduler.

Functions

- void [vConfigureScheduler](#) ([TimelineConfig_t](#) *cfg)
Initializes the time-triggered scheduler.
- void [vTaskSetTimelineConfig](#) ([TaskHandle_t](#) xTask, [uint32_t](#) ulStart, [uint32_t](#) ulEnd, [uint8_t](#) type)
Binds timeline configuration to a FreeRTOS task.

Variables

- [SubFrame_t](#) major_frame [[NUM_SUB_FRAMES](#)]
External reference to the sub-frame array defined in [main.c](#).

7.5.1 Detailed Description

Time-triggered scheduler extension for FreeRTOS.

FreeRTOS V202212.00 Copyright (C) 2020 Amazon.com, Inc. or its affiliates. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

<https://www.FreeRTOS.org> <https://github.com/FreeRTOS>

7.5.2 Macro Definition Documentation

MAJOR_FRAME_DURATION

```
#define MAJOR_FRAME_DURATION ( ( TickType_t ) 100 )
```

Ticks per major frame.

MAX_SRT_PER_SUBFRAME

```
#define MAX_SRT_PER_SUBFRAME ( 2 )
```

Maximum soft real-time tasks per sub-frame.

NUM_SUB_FRAMES

```
#define NUM_SUB_FRAMES ( MAJOR_FRAME_DURATION / SUB_FRAME_DURATION )
```

Total sub-frames per major frame.

SUB_FRAME_DURATION

```
#define SUB_FRAME_DURATION ( ( TickType_t ) 10 )
```

Ticks per sub-frame.

7.5.3 Enumeration Type Documentation

TaskType_t

enum [TaskType_t](#)

Task classification for the time-triggered scheduler.

Enumerator

HARD_RT	Hard Real-Time: must finish before its deadline or is terminated.
SOFT_RT	Soft Real-Time: runs in idle gaps, preemptible by any HRT task.

7.5.4 Function Documentation

vConfigureScheduler()

```
void vConfigureScheduler (  
    TimelineConfig\_t * cfg)
```

Initializes the time-triggered scheduler.

Configures and initializes the time-triggered scheduler.

Call once from [main\(\)](#), before [vTaskStartScheduler\(\)](#). Parses *cfg*, creates all FreeRTOS tasks (suspended), and spawns the Frame Manager task that resets the timeline at every major-frame boundary.

Parameters

<i>cfg</i>	Pointer to the global timeline configuration structure.
------------	---

Initializes the time-triggered scheduler.

This function validates the configuration, cleans up any previous test states, spawns the Frame Manager task, and creates all application tasks (initially suspended).

Parameters

<i>cfg</i>	Pointer to the timeline configuration structure.
------------	--

vTaskSetTimelineConfig()

```
void vTaskSetTimelineConfig (
    TaskHandle_t xTask,
    uint32_t ulStart,
    uint32_t ulEnd,
    uint8_t type)
```

Binds timeline configuration to a FreeRTOS task.

Configures timeline scheduling parameters for a specific task.

Writes the timeline parameters directly into the TCB extension fields added by the [tasks.c](#) patch. Call after `xTaskCreate()` for each task whose start/end times must be enforced inside `xTaskIncrementTick()`.

Parameters

<i>xTask</i>	The handle of the target FreeRTOS task.
<i>ulStart</i>	The tick offset inside the major frame when the task should activate.
<i>ulEnd</i>	The tick offset representing the task's deadline.
<i>type</i>	The task classification (HARD_RT or SOFT_RT).

Binds timeline configuration to a FreeRTOS task.

•

Parameters

<i>xTask</i>	The handle of the task to be configured.
<i>ulStart</i>	The scheduled start time within the major frame (in ticks).
<i>ulEnd</i>	The scheduled deadline/end time within the major frame (in ticks).
<i>type</i>	The real-time task type (0 = HARD_RT, 1 = SOFT_RT).

7.5.5 Variable Documentation**major_frame**

```
SubFrame_t major_frame[NUM\_SUB\_FRAMES] [extern]
```

External reference to the sub-frame array defined in [main.c](#).

•

7.6 scheduler.h

[Go to the documentation of this file.](#)

```

00001
00028
00029 #ifndef SCHEDULER_H
00030 #define SCHEDULER_H
00031
00032 #include "FreeRTOS.h"
00033 #include "task.h"
00034
00038 typedef enum
00039 {
00040     HARD_RT,
00041     SOFT_RT
00042 } TaskType_t;
00043
00047 typedef struct
00048 {
00049     uint32_t id;
00050 } SubFrame_t;
00051
00056 #define MAJOR_FRAME_DURATION    ( ( TickType_t ) 100 )
00057 #define SUB_FRAME_DURATION      ( ( TickType_t ) 10 )
00058 #define NUM_SUB_FRAMES          ( MAJOR_FRAME_DURATION / SUB_FRAME_DURATION )
00059 #define MAX_SRT_PER_SUBFRAME    ( 2 )
00061
00068 typedef struct
00069 {
00070     const char      *task_name;
00071     TaskFunction_t   function;
00072     TaskType_t       type;
00073     uint32_t         ulStart_time;
00074     uint32_t         ulEnd_time;
00075     uint32_t         ulSubframe_id;
00076
00077     /* --- Runtime state (written by the scheduler, not by the application) --- */
00078
00079     TaskHandle_t     xHandle;
00080     uint8_t          is_completed;
00081 } TimelineTaskConfig_t;
00082
00086 typedef struct
00087 {
00088     TimelineTaskConfig_t *tasks;
00089     uint32_t              tasks_num;
00090     uint32_t              num_sub_frames;
00091     uint32_t              major_frame_period;
00092     uint32_t              sub_frame_period;
00093
00094     /* Dynamic counters - maintained by the tick hook, initialised to 0. */
00095
00096     uint32_t              current_tick;
00097     uint32_t              current_sub_frame;
00098 } TimelineConfig_t;
00099
00102 extern SubFrame_t major_frame[ NUM_SUB_FRAMES ];
00103
00113 void vConfigureScheduler( TimelineConfig_t *cfg );
00114
00127 void vTaskSetTimelineConfig( TaskHandle_t xTask,
00128                             uint32_t ulStart,
00129                             uint32_t ulEnd,
00130                             uint8_t type );
00131
00132 #endif /* SCHEDULER_H */

```

7.7 tasks.c File Reference

Functions

- **if (pxTimelineState !=NULL)**
Evaluates timeline tasks and enforces time-triggered scheduling constraints. Wakes up pending tasks and forcefully suspends tasks missing their deadlines.
- **traceRETURN_xTaskIncrementTick (xSwitchRequired)**
- **taskSELECT_HIGHEST_PRIORITY_TASK ()**
- **traceTASK_SWITCHED_IN ()**
- **void vTaskSetTimelineConfig (TaskHandle_t xTask, uint32_t ulStart, uint32_t ulEnd, uint8_t type)**
Configures timeline scheduling parameters for a specific task.

Variables

- [TimelineConfig_t * pxTimelineState](#)
Global state for the timeline scheduler configuration.
- [TaskHandle_t xFrameManagerHandle](#)
Handle for the Frame Manager Task.
- [uint32_t ulStart_time](#)
Start time of the task within the major frame (in ticks).
- [uint32_t ulEnd_time](#)
Deadline or end time of the task within the major frame (in ticks).
- [uint8_t ucTaskType](#)
Task type identifier (0 = HARD_RT, 1 = SOFT_RT).
- [uint8_t is_completed](#)
Flag indicating if the task has completed its execution in the current frame.
- [tskTCB](#)
- [return xSwitchRequired](#)

7.7.1 Function Documentation

if()

```
if (
    pxTimelineState != NULL)
```

Evaluates timeline tasks and enforces time-triggered scheduling constraints. Wakes up pending tasks and forcefully suspends tasks missing their deadlines.

taskSELECT_HIGHEST_PRIORITY_TASK()

```
taskSELECT_HIGHEST_PRIORITY_TASK ()
```

traceRETURN_xTaskIncrementTick()

```
traceRETURN_xTaskIncrementTick (
    xSwitchRequired )
```

traceTASK_SWITCHED_IN()

```
traceTASK_SWITCHED_IN ()
```

vTaskSetTimelineConfig()

```
void vTaskSetTimelineConfig (
    TaskHandle_t xTask,
    uint32_t ulStart,
    uint32_t ulEnd,
    uint8_t type)
```

Configures timeline scheduling parameters for a specific task.

Binds timeline configuration to a FreeRTOS task.

- **Parameters**

<i>xTask</i>	The handle of the task to be configured.
<i>ulStart</i>	The scheduled start time within the major frame (in ticks).
<i>ulEnd</i>	The scheduled deadline/end time within the major frame (in ticks).
<i>type</i>	The real-time task type (0 = HARD_RT, 1 = SOFT_RT).

Configures timeline scheduling parameters for a specific task.

Writes the timeline parameters directly into the TCB extension fields added by the [tasks.c](#) patch. Call after `xTaskCreate()` for each task whose start/end times must be enforced inside `xTaskIncrementTick()`.

Parameters

<i>xTask</i>	The handle of the target FreeRTOS task.
<i>ulStart</i>	The tick offset inside the major frame when the task should activate.
<i>ulEnd</i>	The tick offset representing the task's deadline.
<i>type</i>	The task classification (HARD_RT or SOFT_RT).

7.7.2 Variable Documentation**is_completed**

```
uint8_t is_completed
```

Flag indicating if the task has completed its execution in the current frame.

pxTimelineState

```
TimelineConfig_t* pxTimelineState [extern]
```

Global state for the timeline scheduler configuration.

Global pointer to the timeline state.

Global state for the timeline scheduler configuration.

Set once by [vConfigureScheduler\(\)](#) and then read (never written) by the tick-hook logic inside [tasks.c](#). Declared extern there so the linker connects both translation units.

tskTCB

```
tskTCB
```

ucTaskType

```
uint8_t ucTaskType
```

Task type identifier (0 = HARD_RT, 1 = SOFT_RT).

ulEnd_time

```
uint32_t ulEnd_time
```

Deadline or end time of the task within the major frame (in ticks).

ulStart_time

```
uint32_t ulStart_time
```

Start time of the task within the major frame (in ticks).

xFrameManagerHandle

```
TaskHandle_t xFrameManagerHandle [extern]
```

Handle for the Frame Manager Task.

Handle of the Frame Manager task.

Handle for the Frame Manager Task.

The tick hook resumes it at every major-frame boundary (now == 0) so it can reset HRT tasks. Also declared extern in [tasks.c](#).

xSwitchRequired

```
return xSwitchRequired
```

7.8 test_scheduler.c File Reference

Automated regression test suite for the timeline-based scheduler.

```
#include "test_scheduler.h"
#include "uart.h"
#include <string.h>
```

Functions

- void [vTestLogReset](#) (void)
Resets the event log array and event counter.
- void [vTestLogEvent](#) ([EventType_t](#) eType, const char *pcName, [TickType_t](#) xTick, [uint32_t](#) ulFrameTick)
Logs a test event from task context.
- void [vTestLogEventFromISR](#) ([EventType_t](#) eType, const char *pcName, [TickType_t](#) xTick, [uint32_t](#) ulFrameTick)
Logs a test event from an Interrupt Service Routine (ISR).
- void [vRunAllSchedulerTests](#) (void)
Bootstraps the automated regression test suite.

Variables

- volatile [TestEvent_t](#) xTestEventLog [[TEST_MAX_EVENTS](#)]
Shared event log storing chronological test events.
- volatile [uint32_t](#) ulTestEventCount = 0
Current number of recorded events.
- volatile [TickType_t](#) xLastFrameResetTick = 0
The absolute tick count when the last major frame reset occurred.

7.8.1 Detailed Description

Automated regression test suite for the timeline-based scheduler.

Test catalogue:

1. HRT Task Completes Within Deadline: Verifies tasks finish within limits.
2. HRT Task Deadline Miss / Kill: Verifies overrunning tasks are killed.
3. SRT Task Runs Only During Idle Time: Verifies SRT respects HRT windows.
4. SRT Task Preempted by HRT Task: Verifies HRT immediately preempts SRT.
5. Major Frame Repeats Deterministically: Verifies cycle jitter is ≤ 1 .
6. Overlapping HRT Windows (Stress): Verifies overlap handling without crash.
7. Minimal Time Gap (Edge Case): Verifies 0-tick gaps function seamlessly.
8. NULL / Empty Configuration: Verifies robust invalid-input handling.

7.8.2 Function Documentation

vRunAllSchedulerTests()

```
void vRunAllSchedulerTests (
    void )
```

Bootstraps the automated regression test suite.

Bootstraps and executes the regression test suite.

Creates the supervisor task that sequences all tests. Must be called from `main()` before starting the FreeRTOS scheduler.

Bootstraps the automated regression test suite.

Creates the test supervisor task and returns immediately. All output goes to UART in the format:

```
=====
Timeline Scheduler - Regression Test Suite
=====
Test 1 - HRT Task Completes Within Deadline : PASSED
Test 2 - HRT Task Deadline Miss / Kill      : PASSED
...
=====
Results: 7 / 8 tests PASSED
=====
```

vTestLogEvent()

```
void vTestLogEvent (
    EventType_t eType,
    const char * pcName,
    TickType_t xTick,
    uint32_t ulFrameTick)
```

Logs a test event from task context.

Appends one event to the log (ISR-safe via taskENTER/EXIT_CRITICAL).

Parameters

<i>eType</i>	Type of the event (e.g., EVT_TASK_START).
<i>pcName</i>	Name of the associated task.
<i>xTick</i>	Absolute system tick count.
<i>ulFrameTick</i>	Legacy parameter, overridden internally.

Logs a test event from task context.

Parameters

<i>eType</i>	The type of the event.
<i>pcName</i>	The name of the task associated with the event.
<i>xTick</i>	The absolute tick count when the event occurred.

<i>ulFrameTick</i>	Legacy/Unused relative frame tick.
--------------------	------------------------------------

vTestLogEventFromISR()

```
void vTestLogEventFromISR (
    EventType_t eType,
    const char * pcName,
    TickType_t xTick,
    uint32_t ulFrameTick)
```

Logs a test event from an Interrupt Service Routine (ISR).

Appends one event to the log from an Interrupt Service Routine (ISR).

Parameters

<i>eType</i>	Type of the event.
<i>pcName</i>	Name of the associated task.
<i>xTick</i>	Absolute system tick count.
<i>ulFrameTick</i>	Legacy parameter, overridden internally.

Logs a test event from an Interrupt Service Routine (ISR).

Parameters

<i>eType</i>	The type of the event.
<i>pcName</i>	The name of the task associated with the event.
<i>xTick</i>	The absolute tick count when the event occurred.
<i>ulFrameTick</i>	Legacy/Unused relative frame tick.

vTestLogReset()

```
void vTestLogReset (
    void )
```

Resets the event log array and event counter.

Resets the event log between test runs.

Resets the event log array and event counter.

7.8.3 Variable Documentation

ulTestEventCount

```
volatile uint32_t ulTestEventCount = 0
```

Current number of recorded events.

Global count of recorded test events.

Current number of recorded events.

xLastFrameResetTick

```
volatile TickType_t xLastFrameResetTick = 0
```

The absolute tick count when the last major frame reset occurred.

xTestEventLog

```
volatile TestEvent_t xTestEventLog[TEST_MAX_EVENTS]
```

Shared event log storing chronological test events.

Shared log written by probes, read by verifiers.

Shared event log storing chronological test events.

7.9 test_scheduler.h File Reference

Public API for the timeline-scheduler regression test suite.

```
#include "FreeRTOS.h"
#include "task.h"
#include "scheduler.h"
```

Data Structures

- struct [TestEvent_t](#)
Per-test observation record.

Macros

- #define [TEST_MAX_EVENTS](#) (128)
Maximum number of events recorded per test.

Enumerations

- enum [TestResult_t](#) { [TEST_PASSED](#) = 0 , [TEST_FAILED](#) = 1 }
 - enum [EventType_t](#) {
[EVT_TASK_START](#) = 0 , [EVT_TASK_END](#) = 1 , [EVT_DEADLINE_MISS](#) = 2 , [EVT_FRAME_RESET](#) = 3 ,
[EVT_SRT_PREEMPTED](#) = 4 }
- Indicates the final outcome of a single regression test.*
- Categorization of task and timeline events.*

Functions

- void [vTestLogReset](#) (void)
Resets the event log between test runs.
- void [vTestLogEvent](#) ([EventType_t](#) eType, const char *pcName, [TickType_t](#) xTick, [uint32_t](#) ulFrameTick)
Appends one event to the log (ISR-safe via taskENTER/EXIT_CRITICAL).
- void [vTestLogEventFromISR](#) ([EventType_t](#) eType, const char *pcName, [TickType_t](#) xTick, [uint32_t](#) ulFrameTick)
Appends one event to the log from an Interrupt Service Routine (ISR).
- void [vRunAllSchedulerTests](#) (void)
Bootstraps and executes the regression test suite.

Variables

- volatile [TestEvent_t](#) xTestEventLog [[TEST_MAX_EVENTS](#)]
Shared log written by probes, read by verifiers.
- volatile [uint32_t](#) ulTestEventCount
Global count of recorded test events.

7.9.1 Detailed Description

Public API for the timeline-scheduler regression test suite.

Usage

Call [vRunAllSchedulerTests\(\)](#) from [main\(\)](#) BEFORE [vTaskStartScheduler\(\)](#). It creates one supervisor task that sequences every test, then the scheduler runs normally.

```
int main(void) {
    UART_init();
    vRunAllSchedulerTests(); // <-- registers tests
    vTaskStartScheduler();
}
```

Each test configures its own [TimelineConfig_t](#), calls [vConfigureScheduler\(\)](#), runs for a fixed number of major frames, collects observations through the test_probe hooks, and prints a PASSED / FAILED verdict.

7.9.2 Macro Definition Documentation

TEST_MAX_EVENTS

```
#define TEST_MAX_EVENTS ( 128 )
```

Maximum number of events recorded per test.

7.9.3 Enumeration Type Documentation

EventType_t

```
enum EventType_t
```

Categorization of task and timeline events.

Enumerator

EVT_TASK_START	Task became running.
EVT_TASK_END	Task completed voluntarily.
EVT_DEADLINE_MISS	Task was forcibly terminated.
EVT_FRAME_RESET	Major-frame boundary reached.
EVT_SRT_PREEMPTED	SRT task was preempted by an HRT task.

TestResult_t

```
enum TestResult_t
```

Indicates the final outcome of a single regression test.

Enumerator

TEST_PASSED	The test completed and all invariants held true.
TEST_FAILED	The test violated an expected constraint or invariant.

7.9.4 Function Documentation

vRunAllSchedulerTests()

```
void vRunAllSchedulerTests (
    void )
```

Bootstraps and executes the regression test suite.

Bootstraps the automated regression test suite.

Creates the test supervisor task and returns immediately. All output goes to UART in the format:

```
=====
Timeline Scheduler - Regression Test Suite
=====
Test 1 - HRT Task Completes Within Deadline : PASSED
Test 2 - HRT Task Deadline Miss / Kill      : PASSED
...
=====
Results: 7 / 8 tests PASSED
=====
```

Bootstraps and executes the regression test suite.

Creates the supervisor task that sequences all tests. Must be called from `main()` before starting the FreeRTOS scheduler.

vTestLogEvent()

```
void vTestLogEvent (
    EventType_t eType,
    const char * pcName,
    TickType_t xTick,
    uint32_t ulFrameTick)
```

Appends one event to the log (ISR-safe via taskENTER/EXIT_CRITICAL).

Logs a test event from task context.

Parameters

<i>eType</i>	The type of the event.
<i>pcName</i>	The name of the task associated with the event.
<i>xTick</i>	The absolute tick count when the event occurred.
<i>ulFrameTick</i>	Legacy/Unused relative frame tick.

Appends one event to the log (ISR-safe via taskENTER/EXIT_CRITICAL).

Parameters

<i>eType</i>	Type of the event (e.g., EVT_TASK_START).
<i>pcName</i>	Name of the associated task.
<i>xTick</i>	Absolute system tick count.
<i>ulFrameTick</i>	Legacy parameter, overridden internally.

vTestLogEventFromISR()

```
void vTestLogEventFromISR (
    EventType_t eType,
    const char * pcName,
    TickType_t xTick,
    uint32_t ulFrameTick)
```

Appends one event to the log from an Interrupt Service Routine (ISR).

Logs a test event from an Interrupt Service Routine (ISR).

Parameters

<i>eType</i>	The type of the event.
<i>pcName</i>	The name of the task associated with the event.
<i>xTick</i>	The absolute tick count when the event occurred.
<i>ulFrameTick</i>	Legacy/Unused relative frame tick.

Appends one event to the log from an Interrupt Service Routine (ISR).

Parameters

<i>eType</i>	Type of the event.
<i>pcName</i>	Name of the associated task.
<i>xTick</i>	Absolute system tick count.
<i>ulFrameTick</i>	Legacy parameter, overridden internally.

vTestLogReset()

```
void vTestLogReset (
    void )
```

Resets the event log between test runs.

Resets the event log array and event counter.

Resets the event log between test runs.

7.9.5 Variable Documentation**ulTestEventCount**

```
volatile uint32_t ulTestEventCount [extern]
```

Global count of recorded test events.

Current number of recorded events.

Global count of recorded test events.

xTestEventLog

```
volatile TestEvent_t xTestEventLog[TEST_MAX_EVENTS] [extern]
```

Shared log written by probes, read by verifiers.

Shared event log storing chronological test events.

Shared log written by probes, read by verifiers.

7.10 test_scheduler.h

[Go to the documentation of this file.](#)

```

00001 /*
00002  * FreeRTOS V202212.00
00003  * Copyright (C) 2020 Amazon.com, Inc. or its affiliates. All Rights Reserved.
00004  *
00005  * Permission is hereby granted, free of charge, to any person obtaining a copy
00006  * of this software and associated documentation files (the "Software"), to deal
00007  * in the Software without restriction, including without limitation the rights
00008  * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00009  * copies of the Software, and to permit persons to whom the Software is
00010  * furnished to do so, subject to the following conditions:
00011  *
00012  * The above copyright notice and this permission notice shall be included in
00013  * all copies or substantial portions of the Software.
00014  *
00015  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00016  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00017  * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00018  * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00019  * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
00020  * FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS
00021  * IN THE SOFTWARE.
00022  *
00023  * https://www.FreeRTOS.org
00024  * https://github.com/FreeRTOS
00025  */
00026
00048
00049 #ifndef TEST_SCHEDULER_H
00050 #define TEST_SCHEDULER_H
00051
00052 #include "FreeRTOS.h"
00053 #include "task.h"
00054 #include "scheduler.h"
00055
00056 /* -----
00057  * Test result type
00058  * ----- */
00059
00063 typedef enum
00064 {
00065     TEST_PASSED = 0,
00066     TEST_FAILED = 1
00067 } TestResult_t;
00068
00069 /* -----
00070  * Per-test observation record
00071  * ----- */
00072
00074 #define TEST_MAX_EVENTS    ( 128 )
00075
00079 typedef enum
00080 {
00081     EVT_TASK_START        = 0,
00082     EVT_TASK_END          = 1,
00083     EVT_DEADLINE_MISS    = 2,
00084     EVT_FRAME_RESET       = 3,
00085     EVT_SRT_PREEMPTED     = 4,
00086 } EventType_t;
00087
00095 typedef struct
00096 {
00097     EventType_t eType;
00098     char pcTaskName[ configMAX_TASK_NAME_LEN ];
00099     TickType_t xTick;
00100     uint32_t ulFrameTick;
00101 } TestEvent_t;
00102
00104 extern volatile TestEvent_t xTestEventLog[ TEST_MAX_EVENTS ];
00105
00107 extern volatile uint32_t ulTestEventCount;
00108
00112 void vTestLogReset( void );
00113
00122 void vTestLogEvent( EventType_t eType,
00123                    const char *pcName,
00124                    TickType_t xTick,
00125                    uint32_t ulFrameTick );
00126
00135 void vTestLogEventFromISR( EventType_t eType,
00136                           const char *pcName,
00137                           TickType_t xTick,
00138                           uint32_t ulFrameTick );

```

```

00139
00140 /* -----
00141  * Entry point
00142  * ----- */
00143
00162 void vRunAllSchedulerTests( void );
00163
00164 #endif /* TEST_SCHEDULER_H */

```

7.11 uart.c File Reference

Functions

- The only modification made in uart h is the addition of the The corresponding implementation in the c [va_start](#) (args, s)

Variables

- The only modification made in uart h is the addition of the [function](#)
- The only modification made in uart h is the addition of the The corresponding implementation in the c [file](#)
- while [s](#)

7.11.1 Function Documentation

va_start()

The only modification made in uart h is the addition of the The corresponding implementation in the c [va_start](#) (args, s)

7.11.2 Variable Documentation

file

The only modification made in uart h is the addition of the The corresponding implementation in the c [file](#)

function

The only modification made in uart h is the addition of the [function](#)

s

while s

Initial value:

```
{
    if (*s == '%')
    {
        s++;
        if (*s == 'd' || *s == 'i') {
            UART_print_int( va_arg( args, int ) );
        } else if (*s == 's') {
            char *string = va_arg( args, char* );
            UART_print_string( string ? string : "(null)" );
        }
        s++;
    }
    else
    {
        UART0_DATA = (unsigned int)(*s);
        s++;
    }
}
va_end( args )
```